

Bachelorarbeit

Ansatz zur Optimierung einer CI-Pipeline mit reproduzierbaren lokalen Builds

Ruben Leander Rosenmeyer

Gutachter: Prof. Dr. Peter Thiemann

Betreuer: Dr. Michael Sperber

Albert-Ludwigs-Universität Freiburg

Technische Fakultät

Institut für Informatik

13. Juli 2026

Bearbeitungszeit

13. 04. 2026 – 13. 07. 2026

Gutachter

Prof. Dr. Peter Thiemann

Betreuer

Dr. Michael Sperber

ERKLÄRUNG

Hiermit erkläre ich, dass ich diese Abschlussarbeit selbständig verfasst habe, keine anderen als die angegebenen Quellen/Hilfsmittel verwendet habe und alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten Schriften entnommen wurden, als solche kenntlich gemacht habe.

Darüber hinaus erkläre ich, dass diese Abschlussarbeit nicht, auch nicht auszugsweise, bereits für eine andere Prüfung angefertigt wurde.

Ort, Datum

Unterschrift

Zusammenfassung

Diese Bachelorarbeit beschäftigt sich mit einem Ansatz, der die Gesamtlaufzeit einer Testsuite eines Softwareprojektes reduzieren soll, indem vermieden wird, dass Testfälle, die bereits für den aktuellen Stand der Softwareprojekts evaluiert wurden, erneut ausgeführt werden. Dieser Ansatz bewegt sich in einem Szenario, bei dem eine unbestimmte Teilmenge aller auszuführenden Testfälle bereits durchlaufen wurde. Beispielsweise kann es sich dabei um einen Build-Server handeln, der einen Commit von einem Entwickler erhält, welcher bereits bei sich lokal eine Teilmenge der Tests laufen lassen hat.

Um diesen Ansatz zu realisieren, wird ein Tool erstellt, welches möglichst unabhängig vom verwendeten Testframework entschieden kann, welche Testfälle bereits durchlaufen wurden, und welche nicht. Anhand dieser Evaluation wird dann nur eine Teilmenge der gesamten Testuite ausgeführt, wobei eine insgesamt 100%-ige Abdeckung der Testsuite gewährleistet sein muss.

Dieses Tool soll sich in eine bestehende CI-Pipeline integrieren lassen, auch hier möglichst unabhängig von der verwendeten Software.

Hierfür werden im Folgenden die Anforderungen an ein Projekt, in das das Tool integriert werden kann, beschrieben und erklärt.

Inhaltsverzeichnis

Zusammenfassung	ii
1 Einleitung	1
1.1 Problemstellung	1
1.2 Zielsetzung	2
2 Hintergrund	3
2.1 Anforderungen an das Tool	3
2.2 Anforderungen an ein Testausgabeformat	3
2.3 Vergleich von Testausgabeformaten	5
2.3.1 JUnit XML	5
2.3.2 Test Anything Protocol	5
2.3.3 JSON basierte Formate	6
2.3.4 Andere Formate	6
2.4 Vergleich	6
2.5 JUnit XML im Detail	6
2.6 git notes	6
3 Umsetzung	7
3.1 Anforderungen an ein Projekt	7
3.1.1 Deterministische builds - 'it works on my machine' . .	7
3.1.2 Benötigte Schnittstellen	7

3.2	Anforderungen an ein Framework	8
3.3	Einschränkungen	8
3.4	Codestruktur	9
3.4.1	Parser für JUnit XML	9
3.5	Exemplarischer Ablauf	9
4	Related Work	10
4.1	Andere Tools	10
5	Fazit	11
5.1	Zusammenfassung der Ergebnisse	11
5.2	Möglichkeiten zur Verbesserung	11
	Bibliography	12

1 Einleitung

1.1 Problemstellung

Jedes größere Softwareprojekt benötigt eine Vielzahl von Tests, um die Qualität der Software zu sichern. Dabei werden die einzelnen Tests für gewöhnlich in verschiedene Klassen unterteilt, die verschiedene Abschnitte der Software abdecken. Die meisten gängigen Testframeworks bieten die Möglichkeit, einzelne Testklassen und/oder Testfälle gezielt auszuführen, ohne die restlichen Testfälle dabei mit auszuführen.

Nehmen wir z.B. einen Entwickler, der bei einem großen Softwareprojekt die Funktionalität einer Datenbank-Schnittstelle anpasst. Testfälle, welche die Funktion dieser Schnittstelle abtesten, sind hier sehr relevant - andere Testfälle, die sich mit anderen Abschnitten des Projektes befassen, wie etwa UI-Tests oder Tests für Netzwerkverbindungen, sollten bei einem sauber modularisiertem Projekt mit hoher Wahrscheinlichkeit unabhängig von Änderungen der Datenbank-Schnittstelle unverändert wie vor den Änderungen des Entwicklers ablaufen. Da manche Testfälle viel Zeit für ihre Durchführung in Anspruch nehmen, ist es sinnvoll, bei der Entwicklung nur einzelene relevante Abschnitte zu testen.

Dennoch muss zur Validierung des Codes regelmäßig die gesamte Testsuite durchlaufen werden, damit durch das partielle Testen des Codes keine Fehler unbemerkt im Projekt verbleiben.

Um dies zu gewährleisten, wird für gewöhnlich auf dem Build-Server noch einmal die gesamte Testsuite des Projektes durchlaufen, auch wenn manche Testfälle bereits lokal von Entwicklern ausgeführt wurden, ohne dass sich der zu testende Code in der Zwischenzeit geändert hat. Dadurch wird die Laufzeit der CI-Pipeline unnötig verlängert.

1.2 Zielsetzung

Diese Bachelorarbeit untersucht, ob es möglich ist, die Laufzeit einer Testsuite zu reduzieren, indem eine verlässliche Aussage darüber getroffen wird, welche Teilmenge der Testsuite für den aktuellen Stand des Codes bereits durchlaufen wurde, und welche Teilmenge noch laufen muss, um eine 100%-ige Abdeckung der Testsuite zu erreichen. Dabei soll die Wahl des Testframeworks möglichst keine Rolle spielen.

Dazu werden im Verlauf dieser Arbeit die Anforderungen an ein Tool, sowie Voraussetzungen an ein Projekt beschrieben, in welches das Tool integriert werden kann.

Im Anschluss wird evaluiert, in wie weit das Ziel der Reduktion von abzulaufenden Testcases und die damit einhergehende Reduktion der Laufzeit der gesamten Pipeline erreicht wurde.

2 Hintergrund

2.1 Anforderungen an das Tool

Folgende Anforderungen werden an einen Durchlauf des zu entwickelnden Tools gestellt:

- **Nach Ablauf des Tools wurde jeder Testcase für den aktuellen Stand des Codes genau einmal durchgeführt**
Kein Test wird mehrfach ausgeführt, trotzdem muss die gesamte Testsuite komplett abgedeckt sein.
- **Das Ergebnis der durch das Tool durchgeführten Testläufe ist am selben Ort verfügbar, wie die Ergebnisse der vorherigen Testläufe, auf die das Tool zugreift**

2.2 Anforderungen an ein Testausgabeformat

Jedes Testframework verfügt über eines oder mehrere mögliche Ausgabeformate, welches die Informationen über das Ergebnis eines Testlaufes enthält. Damit ein Tool entwickelt werden kann, welches möglichst frameworkunabhängig funktioniert, muss ein Ausgabeformat gefunden werden, dass von möglichst vielen verschiedenen Testframeworks auf die selbe Art und Weise verwendet wird, damit die Dateien vom Tool über eine Schnittstelle eingelesen

werden können.

Mit Blick auf diese sowie die unter 2.1 beschriebenen Anforderungen an das Tool, ergeben sich folgende Anforderungen an ein potentielle nutzbare Ausgabeformat:

Ein ideales Ausgabeformat für Tests:

- **enthält die Gesamtmenge aller Tests im Projekt**

Dieser Punkt ist essenziell, zum Einen um eine Aussage darüber treffen zu können, ob eine 100%ige Testcoverage erzielt wurde, zum Anderen muss das Tool zum Erzielen einer absoluten Coverage auch auf potentiell jeden vorhandenen Testcase zugreifen können. Dazu muss dem Tool selbstverständlich auch jeder Testcase innerhalb des Projektes bekannt sein.

- **identifiziert jeden Testcase eindeutig**

Je nach Testframework und Implementierung ist es gestattet, mehreren Testcases, die in unterschiedlichen Namespaces leben, mit dem selben Namen zu bezeichnen. Aus dem Ausgabeformat muss in jedem Falle eine eindeutige Zuordnung der Testcases und den Ergebnissen möglich sein.

- **sagt für jeden Testcase einzeln aus, ob er ausgeführt wurde oder nicht**

- **ist ein strukturiertes Format**

- **ist in vielen Frameworks über viele Programmiersprachen hinweg vertreten**

Es wäre utopisch anzunehmen, dass ein strukturiertes Format existiert, welches von jedem existierendem Testframework gleichermaßen

benutzt wird. Dennoch gibt es durchaus gängige Formate, die von unterschiedlichen Testframeworks aus unterschiedlichen Sprachen unterstützt werden, wie folgend in 2.3 untersucht wird.

2.3 Vergleich von Testausgabeformaten

Aufgrund der Vielzahl von verschiedenen Testframeworks und der Vielfältigkeit ihrer Ausgabeformate können im Rahmen dieser Bachelorarbeit nur die verbreitesten Formate untersucht werden.

Beschreiben, woher die Unterschiede kommen (Reporter)

2.3.1 JUnit XML

JUnit XML ist ein XML-basiertes Format, welches ursprünglich für das Java-Testframework JUnit entwickelt wurde. Für JUnit XML existiert kein einheitlicher Standard, d.h. es existiert auch keine beschreibendes .xsd Schemata, wie sonst für XML Formate üblich. Trotzdem wurde JUnit XML von vielen Testframeworks in gleicher oder ähnlicher Form adaptiert. Außerdem existieren viele weitere für das Software-Testing relevante Programme, welche JUnit XML verarbeiten können, beispielsweise viele CI-Systeme. Auf diese Weise hat sich JUnit XML zu einem De-Facto Standard für Ausgabeformate von Testframeworks entwickelt.

2.3.2 Test Anything Protocol

Das Test Anything Protocol (TAP) wurde ursprünglich für die Programmiersprache Perl entworfen, wurde über die Jahre hinweg jedoch auch für Testframeworks in anderen Sprachen weiter entwickelt. Anders als bei JUnit XML wird für TAP ein standardisiertes Schema bereitgestellt und gepflegt (<https://testanything.org/tap-version-14-specification.html>).

2.3.3 JSON basierte Formate

Viele Testframeworks bieten auch allgemeinere Formate wie beispielsweise JSON als Ausgabeformat an. Anders als bei XML existiert für JSON allerdings kein etabliertes Format wie JUnit XML, sondern jedes Testframework bestimmt die Struktur des Outputs selbstständig. Daher existiert eine Vielzahl an unterschiedlichen und einzigartigen Strukturen.

2.3.4 Andere Formate

Selbes Problem wie JSON. Beispielsweise HTML, CSV, Markdown etc

-> kleine Tabelle zum Vergleich der 3 als Übersicht

2.4 Vergleich

Anhand der Evaluation der unterschiedlichen Ausgabeformate in 2.3 wird deutlich, dass es keinen eindeutigen Kandidaten für die Auswahl des Testausgabeformat, welches vom Tool als Input akzeptiert wird, gibt. Ein gemeinsames Problem: zeigen keine nicht-ausgeführten tests an

2.5 JUnit XML im Detail

- Genaue Struktur (unterschiedliche root tags!)
- Auf unterschiedliche Umsetzungen eingehen

2.6 git notes

müssen nicht zwingend verwendet werden, auch andere Kommunikationswege mit Tool könnten etabliert werden. Hier nur Mittel der Wahl, weil ändern Code nicht.

3 Umsetzung

3.1 Anforderungen an ein Projekt

3.1.1 Deterministische builds - 'it works on my machine'

Damit das Tool eine qualifizierte Aussage über die Testergebnisse treffen kann, muss gewährleistet sein, dass alle Tests unabhängig von der Entwicklungsumgebung immer das selbe Testergebnis liefern. Ein Testfall, der auf der Machine eines Entwicklers ein bestimmtes Ergebnis liefert, muss auch zwingend auf einem Build-Server das selbe Ergebnis liefern. Andernfalls könnte eine Diskrepanz zwischen den Testergebnissen zu einer verfälschten Aussage des Tools über den aktuellen Stand der Test führen.

Da Junit XML nicht alle in 2.2 aufgeführten Anforderungen erfüllt, ergeben sich einige weitere Anforderungen an ein Projekt, das das Tool nutzen möchte, um trotzdem das geforderte Ziel zu erreichen.

3.1.2 Benötigte Schnittstellen

test discovery

Damit das Tool die Gesamtmenge aller Tests im Projekt kennt, muss im Projekt ein Script vorhanden sein, welches die die Gesamtmenge aller Tests in einem spezifiziertem Format ausgibt.

-> qualifiedname: Eine Zeichenfolge, die einen Test innerhalb eines Testframeworks eindeutig identifiziert. Beispiele in vers. TestFrameworks bringen!

test execution

Zum Anderen muss dem Tool ein zweites Script zur Verfügung gestellt werden, welches eine Liste von zuvor in 3.1.2 beschriebenen qualifiedNames entgegennimmt, nur diese Tests ausführt, und das Ergebnis (auch wieder JUnit XML) an das Tool zurückmeldet, welches dann diesen Input zusätzlich an die gitnotes anhängt.

Absehen von diesen Anforderungen muss dem Tool lediglich der Pfad zu einem Ordner, der alle relevanten JUnit-XML Dateien enthält, mitgegeben werden.

Zur Vereinfachung dieses Prozesses benutzt das Tool die Datei "config.json", in der die Argumente zum Aufrufen der beiden Skripten, sowie der Pfad zu dem Ordner mit den JUnit-XML Dateien hinterlegt werden.

-> Hier Diagram mit Interaktion zwischen Skripten/Tool einfügen

3.2 Anforderungen an ein Framework

- kann JUnit XML
- enthält essenzielle Informationen für einen Test (genau ausführen!)
- kann tests einzeln aufrufen

3.3 Einschränkungen

- Annahme: keine deliberate bad actors; keine Authentifizierung
- Annahme: An gitnotes angehängte Dateien spiegeln aktuellen Stand vom Code wieder, keine Veränderung der Codebase im Nachhinein

3.4 Codestruktur

3.4.1 Parser für JUnit XML

- Parser des tools - leerzeilen als standart-separator von gitnotes

3.5 Exemplarischer Ablauf

`git notes add -F "$(realpath code/out/report.xml)"HEAD`

4 Related Work

4.1 Andere Tools

Abgrenzung zu z.B Bazel/Gradle?

5 Fazit

5.1 Zusammenfassung der Ergebnisse

5.2 Möglichkeiten zur Verbesserung

In seiner aktuellen Form kann das Tool nicht entscheiden, ob die XML-Ausgabe der Tests auch tatsächlich aus dem aktuellen Stand des Codes entstammt, oder ob im Nachhinein Änderungen an dem zu testenden Code oder den Tests selber vorgenommen wurden, was die Aussagekraft des Tools stark beeinträchtigt.

Ein anderes Problem ist, dass das Tool alle Tests, die nicht an den letzten Schwung gitnotes anhängt sind, als nicht abgelaufen betrachtet, auch wenn einige Testfälle in einer vorherigen Iteration ausgeführt sein könnten, und die Änderungen, die seither im Code gemacht wurden diese Tests überhaupt nicht betreffen.

Eine Möglichkeit, dieses Problem zu beheben, wäre die Integration eines neuen Tools, welches eine Seletive Regression Testing / Regression Test Selection implementiert, und so dem Tool rückmelden kann, welche Tests tatsächlich neu ausgeführt werden müssen, und bei welchen der vorherige Stand noch gültig ist.

Zusätzliche Unterstützung von TAP wäre realistisch, JSON formate eher unrealistisch.

-> Ekstazi <https://github.com/gliga/ekstazi>

Literaturverzeichnis

